Using GIT

- Introduction
- Installing a Git client
  - Cygwin
  - macOS
    - Standard Git
    - Alternatives
  - Red Hat/CentOS/AlmaLinux
    - TIBCO-built Alternatives
    - Git In A RHEL 7 Software Collection
  - Ubuntu
- Git Client Configuration
  - Identification
  - Operation Defaults
  - Authentication
    - Password And PATs (Personal Access Tokens) For Credentials
- SSH Keys For GitHub
- Small Project Repositories
- First Use
  - Cloning or "Checking Out"
- Common GIT tasks
  - File Attributes and Ignored Files
  - Keyword Expansion (Historical)
- Developing using branches
  - Simple Personal Branch Use
  - Moving Changes Between Branches, Or Not
    - Listing Available Changes From A Branch
    - Ignoring A Change From A Branch
    - Merging branches with multiple commits
- Back porting
- Use Case Examples
- Untracked files
  - Clobber verification
- Commit Notices

More instructions for particular uses of Git are presented in the child pages of this page.

- Best Practices For Change Management
- Git Administration For EP Development
- Git For DTM Development
- Using git.tibco.com

Introduction

For EP development, you should understand how Git repositories work. The best source for that is `git-scm.com`. Git is often called "decentralized," and yet we use the most "central" repository server for Git – go figure.

Git is the standard source control system for projects in the EP system; see Source Code Management – 2015 Transitions for background.

TIBCO uses GitHub as the primary source control repository; product and internal repos belong the tibco organization, which you **must** join; see GitHub for guidelines. There is also a TIBCOsoftware organization for public repositories for use by the wider community – our samples and Maven plugin belong to that organization.

The old repositories for EP products that were created before 2020 are hosted on `git.tibco.com` under `/projects/products/ep`; you should be able to browse them.

Installing a Git client

Git 2.x is the minimum required version. Version 1 (still the default on RHEL 7) will cause you pain.

`sbx maintain tools-from-os` installs the default version of Git for the environment, but unfortunately, on RHEL 7, this is not good enough. RHEL 8, Xcode, and Cygwin offer recent versions of Git.

Cygwin

Install the `git` package; as noted above, `sbx maintain tools-from-os` will do this for you.

macOS

Standard Git

Use the version of `git` supplied with Xcode command line tools. You need to install the command line tools, either with the Xcode GUI, or by downloading the installer package from `developer.apple.com` (which requires registration).

```
git --version
git version 2.30.1 (Apple Git-130)
```

Alternatives

MacPorts and Brew track the new version versions of Git. On Brew: `brew install git` ...

If you want to use `git-svn`, the MacPort of `git` using the spec `git+svn+doc+python27` will work, and also has the benefit of obtaining new versions of both Git and Subversion.

Red Hat/CentOS/AlmaLinux

The `git` package for Red Hat/AlmaLinux 8 is new enough for development purposes. On Red Hat 6 and 7, the standard GIt package is too old (for example, 1.8.3.1 on RHEL 7).

TIBCO-built Alternatives

Look under git sbdeps (here's background) for newer editions. For example, in the US East lab, there is a fairly new version `/mnt/tools/sbdeps/git/git-2.30.1-x86_64-linux-centos7` for CentOS 7 available from US East Linux machines. Outside US East, that particular version would be installed under `/usr/local/sbdeps` with the command sudo `/usr/local/bin/sbx maintain sbdeps --sbdeps-exact-packages git/2.30.0/x86_64-linux-centos7`.

Git In A RHEL 7 Software Collection

Red Hat/CentOS 7 offers a Software Collection for Git – currently `rh-git218` for Git 2.18. If the machine is provisioned for `yum` (and, on RHEL, is provisioned for a subscription), installation can proceed directly to:

```
$ subscription-manager repos --enable rhel-server-rhscl-7-rpms
$ yum install rh-git218
```

Interactive addition for a shell is easy:

```
$ scl enable rh-git218 $SHELL
$ which git ; git --version
/opt/rh/rh-git218/root/usr/bin/git
git version 2.18.1
```

…but the SCL design doesn't seem to have an obvious way to perform this for shell profiles without starting up another shell., which is unfortunate.

Later versions of Red Hat track Git more closely.

Ubuntu

Recent releases of Ubuntu track Git closely, but if needed, the latest versions of Git for various Ubuntu releases can be found at the following PPA:

https://launchpad.net/~git-core/+archive/ubuntu/ppa

Git Client Configuration

The git configuration can be performed with command `git config`. In the following examples, the `--global` modifier really means "for my home directory," so you have to do this one for each home directory you own. So that would mean any machine with its own home directory set (meaning most standalone workstations) requires these steps, but only once for any set of Unix machines that share a home directory (lucky you).

The stored file for this information is in a simply formatted file called `.gitconfig` in your home directory, so you can also copy that file to any of your home directories.

The one thing to watch out for is that some configuration options are not accepted by older versions of Git. If you are sharing a `.gitconfig` file that was used with Git 2, it might contain elements that Git 1 will not like. The Git pull.rebase parameter has also evolved over the source of Git 2.

Identification

Identity yourself as a TIBCO developer:

```
git config --global user.email hacker@tibco.com
git config --global user.name "Wolfgang Hacker"
```

If you are using Git to contribute to other repositories not under TIBCO control, you can supply a different email option on the `git clone` command line, as in `--config user.email hacker@gitmongers.com`.

Operation Defaults

Set a sane push default; this is effective for Git 2:

```
git config --global push.default simple
```

This sets up pull to rebase by default, which avoids spurious merge messages when you take changes from `git.tibco.com`:

```
$ git config --global pull.rebase merges # (or preserve for older versions of Git 2) does not touch local merge
commits
$ git config --global branch.autosetuprebase always
```

Git can track moving files when merging, but it sometimes needs a little configuration to do so. This will help, especially when taking late changes from sources that were reorganized from StreamBase Subversion:

```
$ git config --global merge.renamelimit 6000
```

Git's default style of presenting a conflict shows just the two sides of the conflict. If you want to also see the common content before the two branches diverged, you use the `diff3` style:

```
$ git config --global merge.conflictStyle diff3
```

Authentication

GitHub supports SSH authentication, and you might find that most convenient for your style of work.

The next step is to set things up so that you aren't typing your password all the time.

Password And PATs (Personal Access Tokens) For Credentials

On macOS, you can save your password in the Keychain. When your password changes you can update it directly using the **Keychain Access** application, or even easier, just browse the https://git.tibco.com/git/ Web page, enter your new password, and respond with "Update" when you are asked whether to update the password for this web site or not.

```
git config --global credential.helper osxkeychain
```

On Linux, here's how to do something similar.

You can also specify your TIBCO password and GitHub personal access token for authentication to git.tibco.com and GitHub in your `~/.netrc` file:

Is this really better than SSH keys?

```
machine git.tibco.com login tibco-login password tibco-password
machine github.com login TIBCOuser password PAT-from-github
```

When using a PAT for GitHub for the first time, you'll be asked to visit a URL in a web browser, which might involve further authentication. But you shouldn't have to do that on subsequent uses.

SSH Keys For GitHub

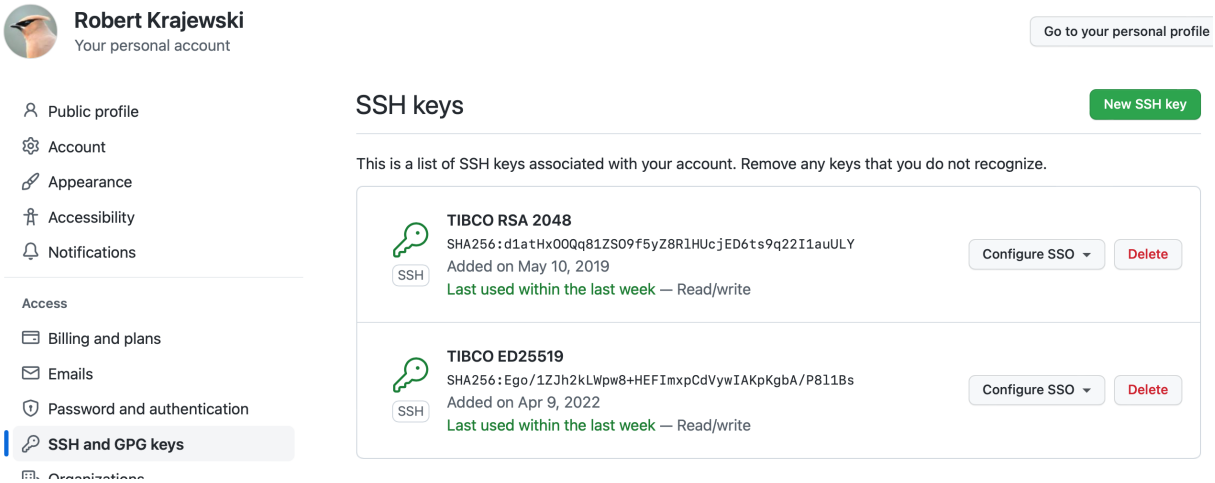
If you have a TIBCO SSH key with a password, you can use that as credentials for your TIBCO GitHub account. You need to use Git URLs with a `git:` scheme, or, for `sbx`, usage is controlled by a preference file in your home directory. Why might you want this?

- If you `ssh` to a machine where your account is set up for GitHub SSH access (like any in the lab cluster), you are ready for authentication, assuming SSH agent forward is in effect; no `~/.netrc` needed. So your PAT doesn't have to be copied around as much.
- You can and should use the same SSH key that you've been using Subversion and lab machine access, so there are fewer elements needed for authentication to TIBCO resources.

To configure:

1. Make sure your SSH public key (of a pair that **has a password**) is added as a public key to your TIBCO GitHub account.
2. Enable that key for SSO/SAML – merely adding it won't work, so hit the button. You will need to do the 2FA dance.
3. For `sbx` use, in your home directory, make a file `~/.TIBdev/github.scm.properties` with the line `connection=ssh`. That means to use ssh for GitHub instead of https and passwords/PATs.

An account with two keys enabled:



Depending on load-balancing on GitHub, you might be asked to verify GitHub's SSH host key for any new home directory instance where you use this for the first time.

If you want to use SSH keys directly from a `git` command, you need to transform the URL if you are given a GitHub https URL. For example, https://github.com/tibco/ep-lvweb.git becomes `git@github.com:tibco/ep-lvweb.git`. If you use `sbx`, it will pick the right protocol given a source name, like `ep-lvweb`. GitHub's web interface will let you choose which scheme to use. Note that GitHub is offering the `git` protocol, but it starts off like `ssh`, so we're just conflating these schemes.

If you were using `sbx checkout` with GitHub PATs, you might want to re-checkout any repos that used https. Before you do that, you should also run `rm -rf ~/.TIBdev/sbx-git/klon/github` so that all reference repositories will start off using `ssh` as well.

Small Project Repositories

> You can create a private repository at GitHub or ask for a new one for the `tibco` organization. Private repositories in the `tibco` organization are discouraged but not prevented, so asking for a new `tibco`-private repository is recommended as soon as it is apparently that a small repository should continue existing. GitHub Pages for the `tibco` organization are private to authenticated users, so if you need Pages functionality, this is a safe way to share things.

We encourage all sharing of resources to use this Git repositories instead of email attachments or posting to a Wiki. A URL to the repository should be posted instead.

First Use

For GitHub, if you haven't done any special setup, you'll need your GitHub id and the personal access token. Desktops may store these details in a secure way - for example Linux KDE defaults to using kwallet.

If you are using macOS Keychain with the git `osxkeychain` authentication helper, you will not be prompted for a password.

EP Development tools that use git may perform many git operations that involve the remote repository. so make sure your authentication issues are taken care of, if only for your own convenience.

Thus, we recommend going over the section above to avoid typing in credentials at all.

Cloning or "Checking Out"

Run `sbx sources` to list our Git repositories, along with descriptions of them.

Use `git clone` to check out a repository. Unlike systems like Subversion and CVS, you check out the whole repository at once. To get started, you can browse to a repository's summary page, like this one on git.tibco.com or this one on GitHub. You can then copy the URL field, cd to a directory where you'd like to put the code, and issue a command like:

```
$ git clone https://github.com/tibco/ep-deveng # repo will be in the ep-deveng directory.
```

If you have `sbx`, you can use `sbx checkout --sources` *`epreponame branch`*, and it will perform a clone in the same way it would check out a part of the StreamBase Subversion tree. The branch name "trunk" is mapped to "master" for git. All EP Git repos, including the samples and Maven plugin repos on GitHub, are supported, and you can check out multiple source sets of the same branch by listing the source names with a a comma, like: `sbx checkout --sources parent,dtm,dtmfilestore,set:streambase,ep-str-productsmoke`

The `sbx` commands for Git work fastest if you already have a "reference repository" around – that's created or updated any time you operate on a branch or check out a particular Git repository. If you want to avoid the wait period for that first operation, which is especially noticeable for large repositories like `dtm` or `qa`, run the command `sbx maintain git-references --git-references all`. This will take a few minutes, but you'll save a lot of time later.

Common GIT tasks

- Find text in this repository: `git grep` *typical-grep-options pattern*
- Pushing a new repository to a new remote; in this case, the TIBCO private project area on GitHub:

  ```
  $ git remote add origin https://github.com/tibco/ep-shiny-and-new.git

  $ git push origin --all # or just a branch name if other branches can be thrown away.
  ```
- Get the remote repository contents without touching local contents: `git fetch`
- Renaming a file or directory: `git mv`
- Display the commit log: git log. The `--follow` option can be used to display the history for files that were moved. The `--stat` option will give you a graphical hint as to extant of the changes for each commit.
- Display the difference between what is staged (with `git add` or a successful `merge` or `cherry-pick`) and HEAD: `git diff --staged`
- Display the differences between the current version and the latest committed version git diff HEAD
- Display the difference between this branch's commit'ed changes and what's on the remote of the branch: `git diff @{u}` – especially useful for seeing how a merge is changing the sources. Pure merge commits don't show any content changes.
- Display the `HEAD` commit contents: git show
- Cleaning out a working copy of all ignored items: `git clean -xdf`
- Checking out or retrieving a file from a particular commit: `git checkout` *`commitish -- path`* Suppose you deleted a file at some commit: you could get it back to what it was last with `git checkout` *`commitish -- path`* – the `~1` means "the commit before the specified one." The file will just be sitting there and you'll have to `git add` it if you want to put it back into repository in your current branch.
- Removing all deletion-caused conflicts from a merge as part of conflict resolution: `git rm $(git diff --name-only --diff-filter=U)` This is useful, but make sure that deletions from the other branch either don't matter, or their old file content has been moved somewhere else. The latter check should not be an issue if you've configured Git for file rename following as recommended.
- Removing a local branch: git branch -d <branch name>
- Removing a remote branch: git push origin --delete <branch name> Doing this might require repository owner privileges; ask one if you run into permissions problems.
- Reset to the state of the tracked branch: `git reset --hard @{u}`; that will discard local commits. Cleaning your working copy is recommended after this.
- Migrating files between git repositories without losing history.

Removing references from a remote repository: git push <repository> --delete refs/heads/<branch name>

> ⚠ Only the repository administrator may delete references in a remote repository.

File Attributes and Ignored Files

Make sure that `.gitignore` and `.gitattributes` reflects which files should be ignored for commits, and which Git attributes (such as line-ending conventions and binary status) should apply to which files. If you are starting a new repository, you can look at other existing repositories, which will tend to have a standard baseline and some customizations.

Some development processes need to have an accurate idea of which files are binary, since copyright notices can appear in binary files, but changing that text might corrupt the file. So make sure `.gitattributes` marks such files as binary.

Keyword Expansion (Historical)

From the Git FAQ:

> *Keyword expansion is not recommended. Keyword expansion causes all sorts of strange problems and isn't really useful anyway, especially within the context of an SCM.*

In order to reduce confusion, the following changes should be made to files as they are updated after the git conversion:

- SCCS_FILE_ID() and SCCS_VERS_ID() macros should be removed.
- All RCS/CVS keywords, including $RCSfile, $Revision, $Date, and $Source, should be removed.

Developing using branches

See Git Branching and Tagging Conventions.

Simple Personal Branch Use

An example of using a personal branch to make changes would look like:

Branch example

```
# push will go to remote repository used for clone
git config --global push.default simple
# clone from master repository.
git clone <dtm_url>
# Go to cloned repository
cd dtm
# Create a personal branch with a meaningful topic name (an issue ID is good), and set the upstream branch to
master
git checkout master
git checkout -b personal/olind/FLUENCY-7294
git branch --set-upstream-to=origin/master personal/olind/FLUENCY-7294
# Make changes to files as needed
...

# commit to branch
git commit -a ...
# Rebase the branch by doing a pull. This moves your changes to the head, avoiding an unnecessary
# "Merge branch 'personal/olind/FLUENCY-7294'" notice when doing the merge below (note: You should
# only do this if you know that nobody has made changes to the files you committed).
git pull --rebase
# Switch to the master, and make sure it has the latest changes.
git checkout master
git pull --rebase
# merge branch with master -- add --squash if you want to render the changes down to one commit
# and --log to that so your commit messages from the branch should be included.
git merge personal/olind/FLUENCY-7294
# push changes to remote repository
git push
# Remove branch (if no longer needed) -- you might need to be a repository owner to do this.
git branch -d personal/olind/FLUENCY-7294
```

Before merging the branch, the master is refreshed using `--rebase`. This is used so that your changes committed to the branch will be applied after any other revisions that have taken place. Doing this avoids the "Merge branch 'master'" step you otherwise may need to do. Note that rebase should only be done if you know that you have no conflicts with master, otherwise the merge may get messy.

You should only delete your old personal branches. Branches for release development should not be deleted because they are important parts of the history of the project and can be used to start hotfixes or patches in a controlled manner. Shared project branches may be deleted if the project manager sees fit.

Moving Changes Between Branches, Or Not

Once there are a few long-lived branches in a repo, you might need to move changes between them, or you might need to ignore certain changes from those branches. For example, a new mainline branch would normally take all the changes of a maintenance branch for a older release, except for changes to version numbers for that older release, since such changes don't make any sense on the "new code" branch.

When we talk about branches here, we mean long-lived branches that exist on `origin` – that is, they are in a repository on git.tibco.com and are there because they represent an important line of development that needs to be preserved.

In StreamBase development, this is normally accomplished with the `sbx mergeout` command, and is facilitated by the fact that `sbx` can figure out the default merge source of the current branch. There is no such convention in git (nor in raw Subversion, except for the parent branch, which is not always the right answer). Instead, you must deal with listing commits and using `git merge` with more explicit options.

In order to keep merge options sane, we will merge instead of rebase, because all the changes we'll be working with have already been seen on the server at git.tibco.com, and are part of a commit chain.

The closest equivalent to merging just a change from another branch, where in Subversion you'd use `-c` or `-r` options to `svn merge`, is command `git cherry-pick`. Note that cherry-pick does not follow file renames or moves.

Listing Available Changes From A Branch

This is done with the `git log` command, but it's not obvious how you get useful output. You should get into the checkout of the repo where you want changes to go, and issue a command like:

```
$ git log HEAD..origin/_source-branch_ --stat
```

The `origin/`*source-branch* notation will consider changes from source branch on the origin. The `--stat` option will show the commit information and the list of changed files.

The `git diff` command can be used to get a list of the changed files in a branch compared to another branch. For example from a work branch, this command will display all changes that are not in **master**.

```
git diff --name-status master
```

Ignoring A Change From A Branch

Normally, this is done because you want to ignore version-specific changes (like changing a version of the product) on the source branch, but you want to take other changes like bug fixes and enhancements.

This is done with the "ours" strategy, like `git merge -s ours` *`ref`*, where *ref* is another branch or the newest unmerged commit from that branch that you want to ignore. Note you should first merge older commits behind (older than) *ref* first if you *want* their changes, then skip the skippable *rev* with `-s ours`.

For `sbx` in Subversion, you would issue a command like `sbx mergeout merge -c 324945 -m 'Ignore version bump.'`

(TBS: tutorial on making a branch "catch up" with some changes we want to ignore. It's best to avoid the awkward by *immediately* no-op merging such changes up as soon as possible so that other people do not have to deal with them.)

Merging branches with multiple commits

There are some cases where a branch is long-lived and contains multiple commits. In general, this should try to be minimized and new feature development should be done in the main development branch - **master**. However, when re-architecting existing code it may not be feasible to work this way since incremental changes would be disruptive. These steps can be used to create a single commit when merging back into **master**.

1. Checkout the work branch.
2. Merge `master` into the work branch.
3. Create a patch file using `git diff master > ../my-work-branch.patch.`
4. Create and checkout a new branch off of `master` in which to apply the patch, for example `my-work-patch-branch`.
5. Apply the patch file using `git apply ../my-work-branch.patch` in the `my-work-patch-branch`.
6. Validate the patch by running tests.
7. Create a summary commit message for the work.
8. Commit the patch files in `my-work-patch-branch`.
9. Checkout `master`.
10. Merge `my-work-patch-branch` branch into `master`.

This technique provides a single commit into the development branch that can be easily cherry-picked to maintenance branches or backed out with a single revert. Very handy.

Back porting

We work on the master for normal development, but sometimes fixes made on master need to be back ported to a maintenance branch. Collectively, master and sb/main branches are considered main branches. Backporting uses a Git operation called *cherry-picking*.

Don't move a change from one main branch to another unless you're absolutely sure that change works where it was first checked in. Unless there's something that is *immediately* blocking *all* milestones (with respect to the main branch versions), just let your change be run again the milestones, and use Jenkins as the final measure of whether you fixed something. Then you can be confident that you're propagating fixes instead of bugs.

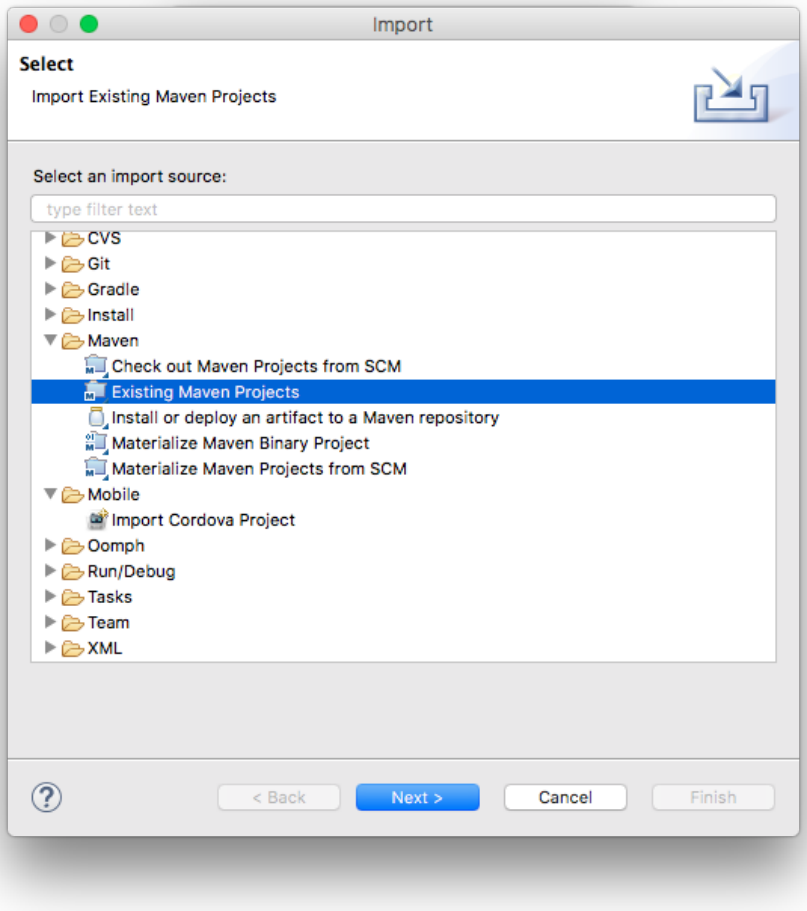
Note that Git cherry picking doesn't deal with moved files between branches. That means, if you are working on a fix that applies to an older main branch whose files might have been refactored later, it's best to do the work on the older branch, confirm it, then `git merge` to the newer branch. The intention for the auto-mergeout process is to free developers from having to perform that git merge unless there's a conflict or build problem.

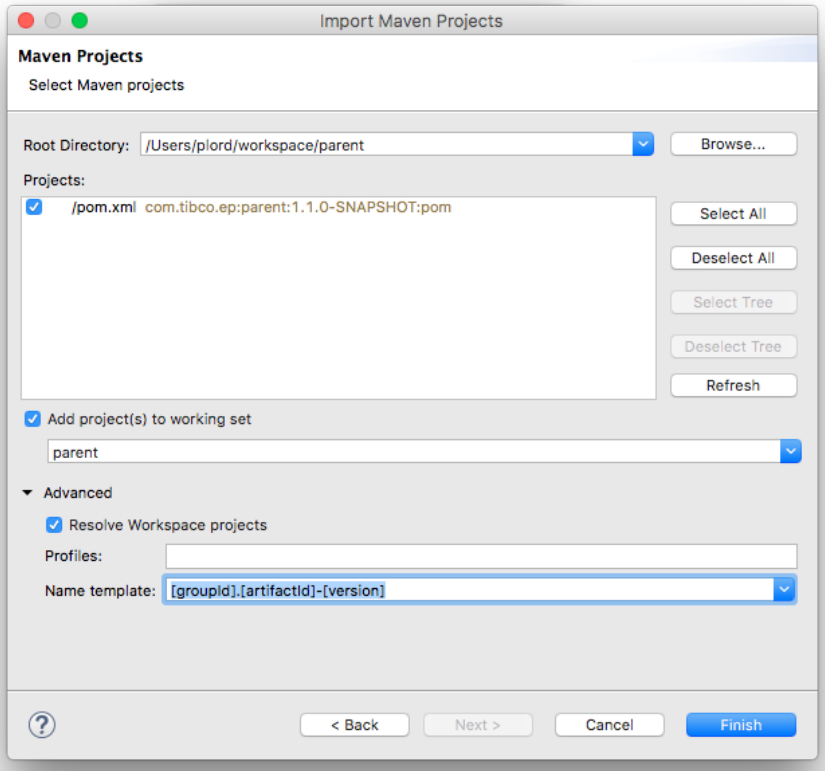
Perform an ordinary commit on one main branch **only**. Then cherry-pick (or merge) for other main branches. This allows people and processes to track such changes.
When you cherry pick, let Git add a comment saying that a commit is a cherry-pick. The command line (`-x`) and eGit have options for this.

Use Case Examples

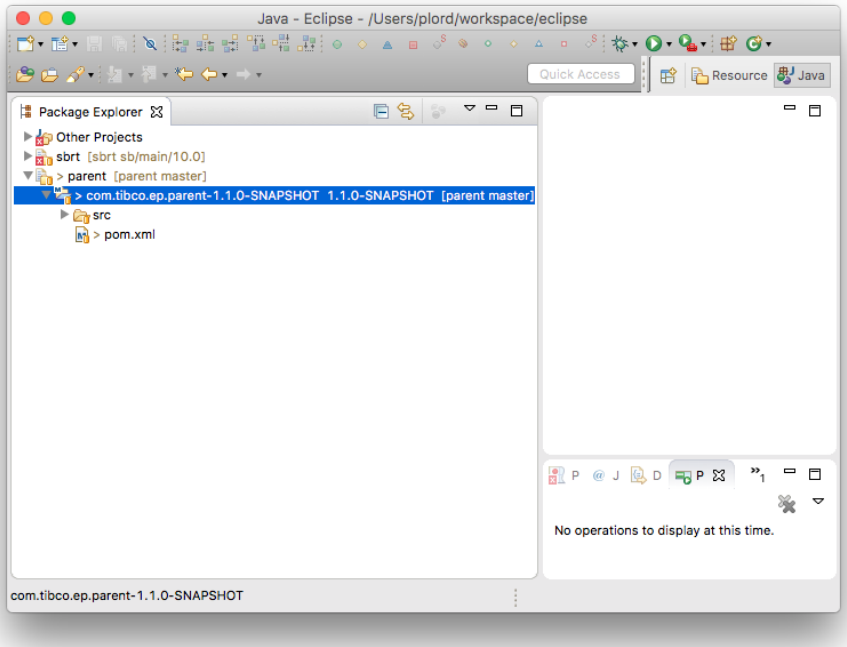
The following uses repository parent as an example with a branch sb/main/10.0, but the same applies to all codelines of course.

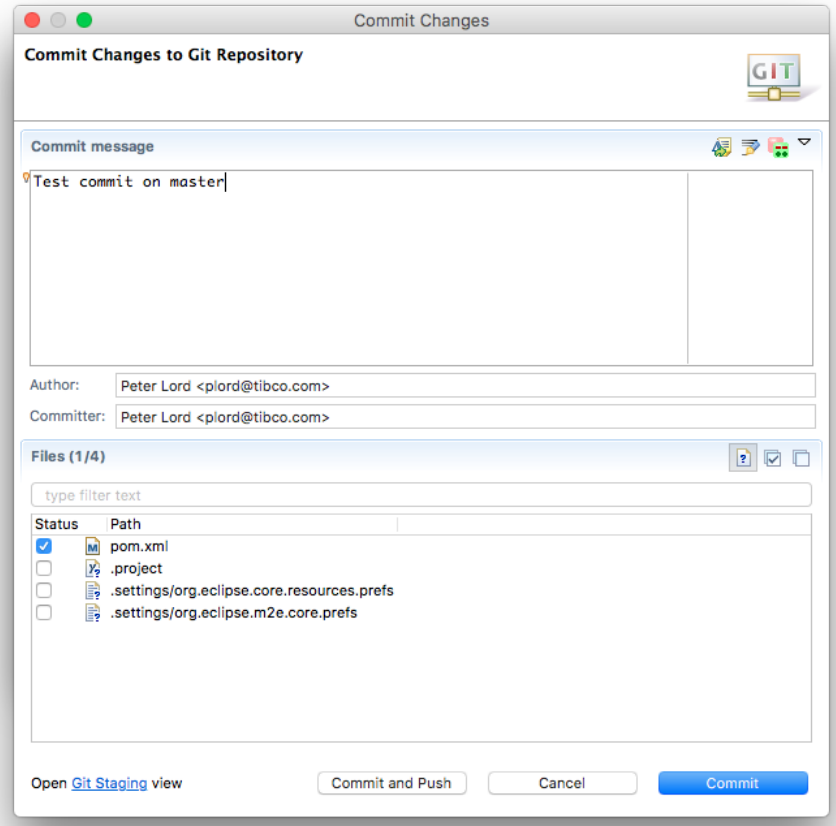
Step	Command-line	Eclipse / Studio

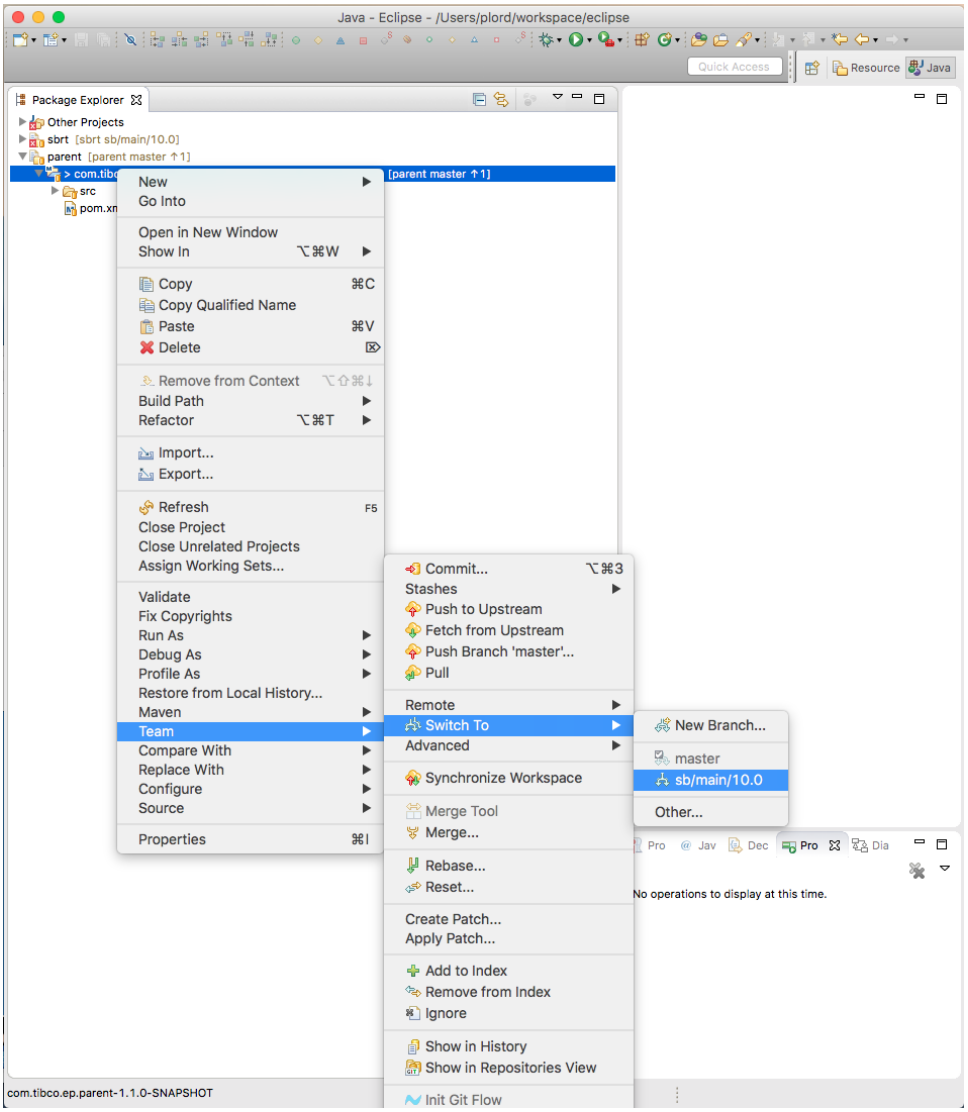
Initial clone and fetch branches	`$ git clone https://git.tibco.com/git/product/ep/parent.git`
	`$ cd parent`
	`$ git fetch`
		I tend to do the clone from the command-line then import.
		
		Menu -> Import and select Existing Maven Projects
		
		 
		
		Browse to file system location, add a working set name and choose a name template :

<table>
<tr>
<td></td>
<td></td>
<td> 
</td>
</tr>
<tr>
<td>Validate that you are working on master</td>
<td><pre>$ git branch
* master
  sb/main/10.0</pre>
The starred branch shows the current one.</td>
<td>Verify that Team -> Switch To lists master and branch

The current one is shown in package explorer :

 
</td>
</tr>
</table>

Work on **master** as needed and commit	`$ git commit`	Team -> Commit  Press Commit

Switch to the branch.
 	
Note that you should clean the workspace (eg mvn clean)
 	
as required by the source code.
	``
	$ git checkout sb/main/10.0
	Branch sb/main/10.0 set up to track remote branch sb/main/10.0 from origin.
	Switched to a new branch 'sb/main/10.0'
	``
		Team -> Switch To -> branch
		 

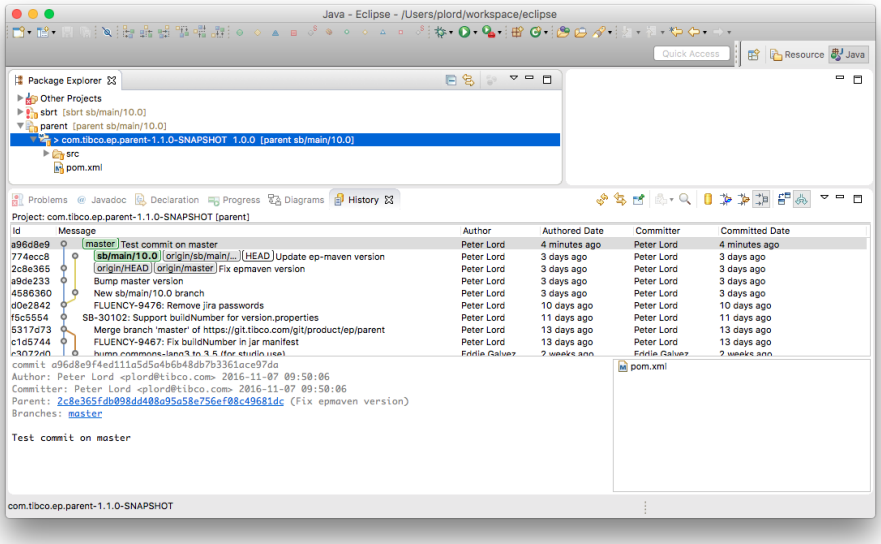
Add the latest commit from **master** to the current branch

and commit.

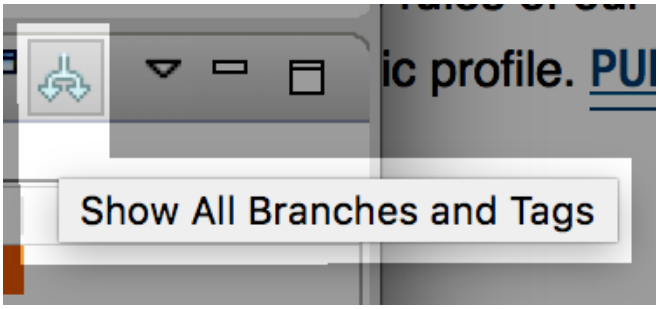
```
$ git cherry-pick -x master
[sb/main/10.0 995a832] Test commit on master
 Date: Mon Nov 7 09:46:02 2016 +0000
 1 file changed, 1 insertion(+)
```

Note there are many options available - see https://git-scm.com/docs/git-cherry-pick The `-x` option adds a comment marking the commit as a cherry-pick, which is a useful clue for both people and processes.

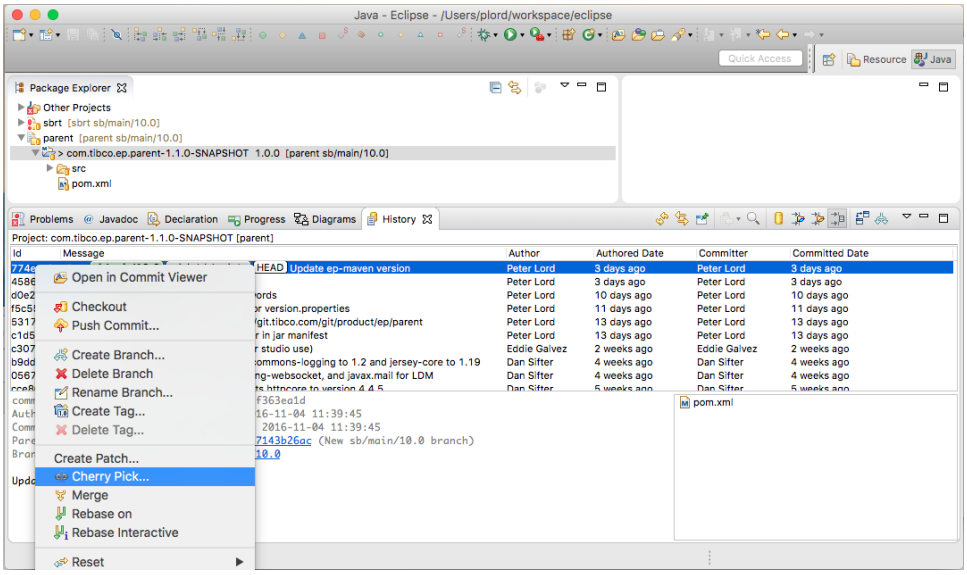
Team -> Show in History

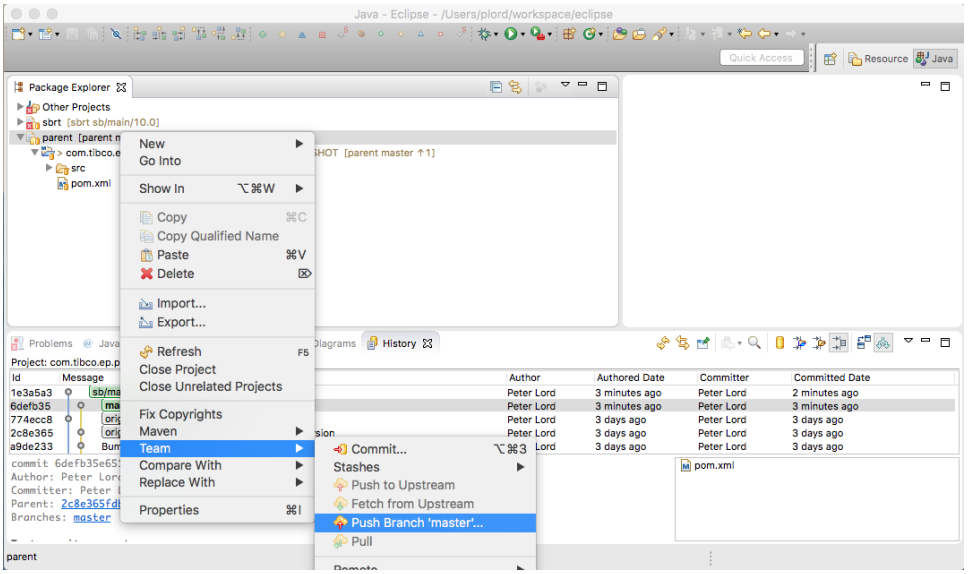


Make sure "Show All Branches and Tags" button is displayed. Look for the branching icon around the history window's toolbar:



Select commit you want to apply to the branch and Menu -> Cherry Pick.



After validating (ie run test cases as needed) both master and branch, push to origin	`$ git push`	Team -> push to master 

Untracked files

The build system places generated artifacts in the working directories, resulting in a large number of "untracked files". Git allows you to ignore artifacts by using a .gitignore file checked into the repository. The dtm repository has been updated to add .gitignore files to all dtm/native/product directories (i.e. the leaf directory for each product), as an example, native/javaservice/libsrc/.gitignore contains:

```
jniwrapper.cpp
```

since this is a generated file. Changes to these directories are pretty rare, but any updates may require modifications to the local .gitignore file.

The .gitignore files can exist anywhere in the project's file hierarchy. If at the root directory, files in all subdirectories will be scanned for matching patterns. Because of this, care should be made so that files in parent directories are carefully chosen. As an example, using "**/build/**" in dtm/.gitignore will screen out all files in dtm/native/build. For this reason, any .gitignore file checked into a repository that is not at a leaf directory should be reviewed before committed.

The default `.gitignore` used in the DTM repository is:

```
**/.settings/**
**/docbook_build*/**
**/BUILD/**
**/devobj/**
**/devlib/**
**/devbin/**
**/prodobj/**
**/prodlib/**
**/prodbin/**
**/bin/**
**/lib/**
**/target/**
**/devtest/**
**/prodtest/**
**/test_DEVELOPMENT/**
**/test_PRODUCTION/**
DISTRIB/
.classpath
.project
TESTLOG.DEVELOPMENT.*
TESTLOG.PRODUCTION.*
```

As mentioned, any changes to this file should be reviewed before being committed.

Clobber verification

Since many of the build and test artifacts are no longer reported when "git status" is run, it is important that "git status --ignored" is executed to verify that "mvn clean" removed all generated product and test artifacts.

Commit Notices

There are a lot of postings on what git commit messages should look like (google "git commit message best practices"). We follow the seven rules outlined in http://chris.beams.io/posts/git-commit. Commits that are associated with an issue (most should be) must have the issue number as the prefix in the first line of the commit, for example:

```
SB-33871: reduce ssh usage in node command
```

Commits that are associated with changes because of a Crucible review should include the Crucible review identifier as the prefix in the first line of the commit, for example:

```
SBCR-611: SSO review updates
```